

JUNIOR PENTESTER PREP

Password Attacks & Credential Access

Field Notes: Hash Cracking, Spraying, Kerberoasting, NTLM Relay & Responder

📖 Compiled for: Goraya — BS Software Engineering, GCUF

🕒 Track: Backend Development + Offensive Security

📅 Format: Senior Pentester → Junior Analyst mentoring notes

📁 Scope: Credential Access (MITRE ATT&CK Tactic TA0006)

⚠️ **Authorized use only.** Every technique in this document is intended strictly for authorized penetration tests, CTFs, and isolated lab environments (your own VMs, HTB/THM, or a signed engagement scope). Running these tools against systems you do not own or do not have written permission to test is illegal in most jurisdictions.

Contents

1	Mental Model: Online vs. Offline Attacks	Module 1
2	Hashing, Hash IDs & Salting	Module 2
3	Cracking Engines: Hashcat & John the Ripper	Module 3
4	Wordlists, Rules & Mask Attacks	Module 4
5	Password Spraying & Credential Stuffing	Module 5
6	Kerberoasting & AS-REP Roasting	Module 6
7	LLMNR/NBT-NS Poisoning with Responder	Module 7
8	NTLM Relay Attacks	Module 8
9	Attacking Behind Load Balancers & WAFs	Module 9
10	Master Decision Tree & Cheat Sheet	Module 10
11	Extended Tool Arsenal	Module 11
12	Blue Team / Defensive Playbook	Module 12
13	Reporting Template	Module 13
14	Quiz Answer Key (Lessons 1, 6, 7, 8)	Module 14

Mental Model: Online vs. Offline Attacks

Before you touch a single tool, you need one habit: **classify the situation before you act**. Junior testers fail interviews and labs not because they don't know Hashcat syntax, but because they can't tell you *why* they picked Hashcat over Hydra. Fix that here.

1.1 The two universes of password attacks

ONLINE ATTACK

```
-----
Attacker --guess--> Live service
Live service --reply--> allow/deny
Server SEES every attempt
```

OFFLINE ATTACK

```
-----
Attacker --has--> Hash / ticket / capture
Attacker cracks it locally, no traffic
Server sees NOTHING
```

Property	Online	Offline
Attacker needs	Network access to the login service	A hash, ticket, or captured exchange
Speed	Slow — network + lockout limited	Fast — millions/sec on GPU
Detectable by target?	Yes — every attempt is logged	No — happens on attacker's machine
Example tools	Hydra, Medusa, CrackMapExec/NetExec	Hashcat, John the Ripper
Example techniques	Password spraying, credential stuffing	Dictionary/brute-force cracking, Kerberoasting

SENIOR TIP

Every time you get a new piece of evidence in an engagement, ask: **"Am I about to talk to the server, or am I about to talk to my own CPU/GPU?"** That single question tells you which tool family you're in before you even think about syntax.

1.2 Why offline attacks matter so much

Modern environments throttle online guessing (account lockout, rate limiting, MFA). This is exactly why real-world engagements gravitate toward techniques that produce **offline material** — a captured hash, an NTLMv2 response, or a Kerberos service ticket — because once it's offline, the target's defenses (lockout policy, WAF, rate limiter) are irrelevant. You are no longer knocking on their door; you're working on a copy in your own lab.

Evidence in hand

```
-----
Plaintext hash file -->
List of usernames only -->
Kerberos SPN account -->
Captured NTLMv2 traffic -->
Known creds from a breach -->
```

Attack family

```
-----
Offline cracking (Hashcat/John)
Password spraying (online)
Kerberoasting (online capture -> offline crack)
Responder capture -> offline crack, or relay
Credential stuffing (online)
```

Hashing, Hash IDs & Salting

Added by mentor

The original transcript covered the core "hash and compare" loop well. Two things a junior tester gets tested on constantly were missing: **identifying a hash type** and **understanding salts**. Both go here.

2.1 The hash-and-compare loop (recap)

```

candidate password --> HASH() --> candidate_hash
                        |
                        v
compare candidate_hash == target_hash ?
                        |
yes -----+----- no
      |               |
      |               |
PASSWORD FOUND    try next candidate
  
```

2.2 Identifying an unknown hash

In a real assessment you rarely get told "this is MD5." You get a blob of characters and have to work it out before you can even pick a Hashcat mode.

Hash format (example)	Length	Likely algorithm	Hashcat mode
5f4dcc3b5aa765d61d8327deb882cf99	32 hex chars	MD5	-m 0
aad3b435b51404eeaad3b435b51404ee	32 hex, often all zero LM half	NTLM	-m 1000
\$2a\$10\$N9qo8uLOickgx2ZMRZoMy...	starts with \$2a\$/\$2b\$	bcrypt	-m 3200
\$6\$rounds=...\$...	starts with \$6\$	sha512crypt (Linux /etc/shadow)	-m 1800
admin::N46iSNekpT:.....:....	colon-separated 3 hash fields	NTLMv2 (captured by Responder)	-m 5600
\$krb5tgs\$23\$*user\$REALM\$...	starts with \$krb5tgs\$	Kerberos TGS (Kerberoasting)	-m 13100
\$krb5asrep\$23\$user@REALM:...	starts with \$krb5asrep\$	AS-REP roast hash	-m 18200

Practical: fingerprint a hash automatically

```

hashid # quick guess based on regex pattern
hashcat --identify hash.txt # hashcat's own identifier, more reliable
  
```

2.3 Why "trinity" and "TRINITY" don't produce the same hash

Hash functions are deterministic and sensitive to every byte, including case. This is why wordlists usually include common casing variants (Trinity, TRINITY, trinity1) and why tools support case-mutation rules rather than expecting you to type every variant by hand (Module 4).

2.4 Salting — the concept the transcript skipped

If every system just stored `HASH(password)`, two users with the same password would have an identical stored hash, and a precomputed "rainbow table" of common password hashes could crack an entire database instantly. A **salt** fixes this: a random value is mixed in before hashing, and stored alongside the hash.

```

stored_hash = HASH(salt + password)
stored record = salt : stored_hash
  
```

```

Two users, same password "trinity123", DIFFERENT salts:
user1: salt=8f2a -> HASH(8f2a + trinity123) = AAAA...
user2: salt=91cd -> HASH(91cd + trinity123) = BBBB...
  
```

WHY THIS MATTERS FOR CRACKING

Salting doesn't make a single hash harder to crack (Hashcat still hashes each candidate with the known salt and compares), but it destroys precomputed rainbow tables and means you must crack every hash individually — no shortcuts across a leaked database. bcrypt, sha512crypt and scrypt all bake salting (and deliberate slowness) directly into the format shown in the table above.

Cracking Engines: Hashcat & John the Ripper

3.1 Hashcat command anatomy

```
hashcat -m <hash-mode> -a <attack-mode> hash.txt wordlist.txt

-m 0      # hash type (0 = MD5, 1000 = NTLM, 13100 = Kerberoast, ...)
-a 0      # attack mode (0 = dictionary, 3 = brute-force/mask, 6/7 = hybrid)
--show    # display already-cracked results without re-cracking
-o out.txt # write cracked pairs to a file
-r rule.txt # apply a rule file to mutate the wordlist (Module 4)
```

Full lab walkthrough (safe, local, no target needed)

```
echo -n "trinity" | md5sum          # create a known-answer hash
nano hash.txt                     # paste ONLY the 32-char hash
nano passwords.txt                # admin / password / trinity / ...
hashcat -m 0 -a 0 hash.txt passwords.txt
hashcat -m 0 hash.txt --show
```

3.2 John the Ripper — when to prefer it over Hashcat

Scenario	Better tool	Why
GPU available, huge wordlist/mask job	Hashcat	GPU acceleration, far higher hashes/sec
Odd/legacy format, e.g. zip/rar/pdf/ssh-key passphrases	John	Broadest "format zoo" + built-in <code>*2john</code> extractor scripts
CTF with a weird custom hash	John	Easier to hand-write a custom format
/etc/shadow dump	Either	John's <code>unshadow</code> merges passwd+shadow first

```
# extracting a crackable hash from a protected file (common junior task)
zip2john protected.zip > zip.hash
rar2john protected.rar > rar.hash
ssh2john id_rsa > ssh.hash
john --wordlist=rockyou.txt zip.hash
john --show zip.hash
```

Wordlists, Rules & Mask Attacks

4.1 The standard wordlist arsenal

Source	Best for	Location on Kali
rockyou.txt	General human passwords (14M real leaked passwords)	<code>/usr/share/wordlists/rockyou.txt</code>
SecLists	Everything — usernames, directories, default creds, subdomains	<code>/usr/share/seclists/</code>
CeWL	Custom candidates scraped from the target's own website	install separately / gem
crunch added	Generating every combination for small, known-pattern brute forces	built-in on Kali

```
cewl https://target-website.com -d 2 -m 5 -w custom-wordlist.txt
# -d 2  crawl depth    -m 5  minimum word length  -w output file

crunch 8 8 -t Comp@ny%% -o crunch_out.txt
# generates 8-char candidates following a known company password pattern
```

4.2 Rule-based mutation added by mentor

Real users don't pick "password" — they pick "Password1!". Instead of manually typing every capitalization/number/symbol variant into your wordlist, apply a **rule file** that mutates each base word at crack time.

```
hashcat -m 0 -a 0 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

```
base word: "trinity"
+ best64.rule mutations -->
trinity, Trinity, TRINITY, trinity1, trinity123, Trinity!, ytinirt (reversed), ...
```

4.3 Brute force vs. dictionary vs. mask attack

Attack mode	Hashcat flag	Idea	Cost
Dictionary	-a 0	Try real, likely candidates from a wordlist	Low — fast, high hit-rate on weak passwords
Brute force	-a 3 (no mask)	Try every possible combination blindly	Very high — explodes with length/charset
Mask attack	-a 3 with a mask	Brute force, but constrained to a known pattern	Medium — smart middle ground
Hybrid	-a 6 / -a 7	Wordlist word + brute-forced suffix/prefix	Low-medium — great for "word + 4 digits" policies

```
# You learned the company policy is: Capitalized word + 2 digits + symbol, e.g. Summer24!
hashcat -m 0 -a 3 hash.txt ?u?l?l?l?l?d?d?s
# ?u upper  ?l lower  ?d digit  ?s symbol — this mask targets exactly that pattern

hashcat -m 0 -a 6 hash.txt rockyou.txt ?d?d?d?d
# hybrid: every rockyou word + 4 brute-forced digits (e.g. "summer"+"2024")
```

COMMON JUNIOR MISTAKE

Reaching for pure brute force (`-a 3` with a huge unconstrained mask like `?a?a?a?a?a?a?a`) before trying a dictionary + rules combo. In real assessments, 90%+ of crackable passwords fall to rockyou + best64 rules in minutes. Save brute force for short, known-length hashes (e.g. 4-digit PINs).

Password Spraying & Credential Stuffing

5.1 Spraying vs. brute force vs. stuffing — don't mix these up

Technique	What you have	What you try	Main risk
Brute force / dictionary	One account, or a hash	Many passwords against one target	Account lockout on that one account
Password spraying	Many usernames	One or two common passwords across ALL of them	Lockout across the whole org if you're greedy
Credential stuffing	Real username:password pairs from a breach	The exact same pairs against a different service	Relies on password reuse across sites

5.2 The golden rule: respect the lockout policy

Lockout policy example: 5 bad attempts / 30 minutes lockout window

Spray strategy:

```
try 1 password --> ALL users
wait > lockout window (e.g. 30-45 min buffer)
try next password --> ALL users
repeat
```

WHY THIS MATTERS

Spraying too fast is the #1 way a junior tester locks out an entire company's Active Directory and turns a stealthy engagement into a help-desk incident. Always confirm the lockout threshold (from scoping documents or by testing on a throwaway account) before spraying live infrastructure.

Practical tools

```
# Active Directory / SMB spraying with NetExec (formerly CrackMapExec)
netexec smb 10.10.10.0/24 -u users.txt -p 'Summer2024!' --continue-on-success

# Kerberos-based spraying (quieter, doesn't touch SMB/LDAP as heavily)
kerbrute passwordspray -d corp.local users.txt 'Welcome2026!'

# HTTP login form spraying
hydra -L users.txt -p 'Summer2024!' target.com http-post-form \
"/login:username=^USER^&password=^PASS^:Invalid credentials"
```

5.3 Credential stuffing in practice

You take a breach dump (e.g. from a public leak database, only ever used with explicit written authorization to test whether the client's staff reuse passwords) and replay the exact username:password pairs against the client's login portal.

```
hydra -C leaked_userpass.txt target.com http-post-form \
"/login:username=^USER^&password=^PASS^:Invalid credentials"
# -C takes a combined "user:pass" file rather than separate -L/-P lists
```

Kerberoasting & AS-REP Roasting AS-REP added

6.1 Kerberoasting flow

```

Authenticated domain user (even low-privileged)
|
v
Request a Kerberos service ticket for an SPN-registered account
|
v
Domain Controller encrypts part of the ticket with the SERVICE ACCOUNT's
password hash (NOT your own)
|
v
You receive the ticket -- take it OFFLINE
|
v
Crack it with Hashcat (-m 13100) or John using rockyou/rules
|
v
If the service account's password is weak --> plaintext password recovered

```

WHY THIS WORKS

Any authenticated domain user can request a service ticket for any SPN — no special privileges needed. The security of the whole technique hinges entirely on whether the service account (often a legacy SQL/IIS/backup account with a password set once in 2014 and never rotated) has a strong password.

Practical (isolated AD lab only)

```

# Enumerate SPNs + request tickets, from Linux (Impacket)
GetUserSPNs.py corp.local/juser:Password1 -dc-ip 10.10.10.5 -request

# From Windows, using Rubeus
Rubeus.exe kerberoast /outfile:tgs.txt

# Crack the extracted $krb5tgs$ hash
hashcat -m 13100 -a 0 tgs.txt rockyou.txt -r best64.rule

```

6.2 AS-REP Roasting — the sibling attack

If an account has "*Do not require Kerberos preauthentication*" enabled, you don't even need valid credentials to request its material — you only need the username.

```

Attacker (no valid creds needed, just a username list)
|
v
Request AS-REP for accounts with preauth disabled
|
v
Receive $krb5asrep$ hash --> crack offline with Hashcat -m 18200

```

```

GetNPUsers.py corp.local/ -usersfile users.txt -no-pass -dc-ip 10.10.10.5
hashcat -m 18200 -a 0 asrep.txt rockyou.txt

```

6.3 Kerberoasting vs. AS-REP roasting

	Kerberoasting	AS-REP Roasting
Requires valid domain creds?	Yes	No — just usernames
Targets	Service accounts with an SPN	Accounts with preauth disabled

Hashcat mode	13100	18200
--------------	-------	-------

LLMNR/NBT-NS Poisoning with Responder

7.1 The full attack chain

```

Windows victim mistypes a share, or a stale DNS entry fails
    |
    v
Victim falls back to LLMNR / NBT-NS broadcast: "Who is FILESRV?"
    |
    v
Responder (on the same local segment) answers: "I'm FILESRV"
    |
    v
Victim authenticates to the attacker, believing it's the real host
    |
    v
NTLMv2 challenge-response captured by Responder
    |
    +-----+-----+
    v               v
Offline crack      Relay (Module 8)

```

Practical (isolated lab / your own VLAN only)

```

sudo responder -I eth0 -wrf
# -I interface -w WPAD proxy spoof -r NBT-NS -f fingerprint OS

# captured NTLMv2 hash looks like:
# user::CORP:1122334455667788:AABBCC...:0101000000000000...

hashcat -m 5600 -a 0 captured.txt rockyou.txt -r best64.rule

```

SENIOR TIP

Responder is noisy by design — it answers broadcast traffic for the whole segment. On an engagement, run it in **-A** (analyze-only) mode first to see how much LLMNR/NBT-NS traffic exists before you start actively poisoning, so you don't disrupt the client's legitimate network traffic.

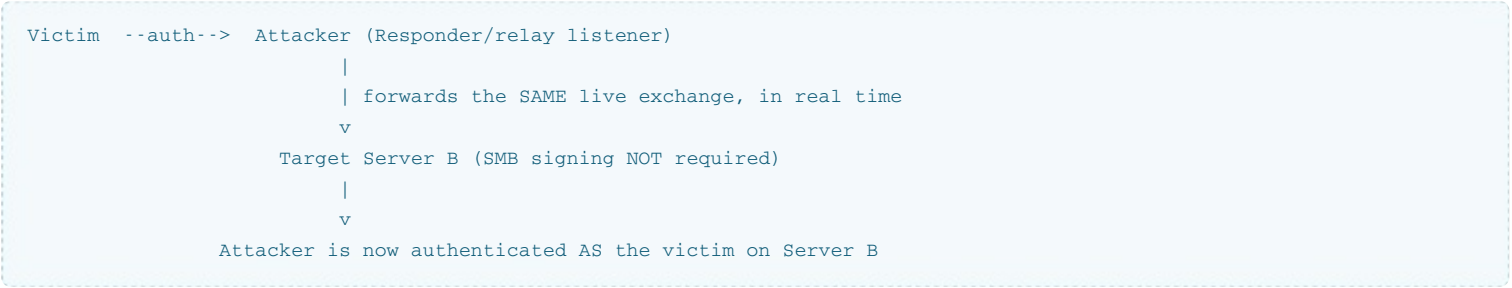
NTLM Relay Attacks

8.1 Crack vs. Relay — the fork in the road

	Crack the capture	Relay the capture
Goal	Recover the plaintext password	Reuse the live authentication session itself
Speed	Depends on password strength — could take forever	Immediate, if SMB signing is disabled on the target
Best when	Password looks weak / short	Password looks strong, but signing is off

8.2 Why relay works: SMB signing

When Responder captures a challenge-response, that material is only valid for a single authentication exchange with a single destination. NTLM relay abuses the fact that **the victim doesn't know or verify which server they're really authenticating to** if SMB message signing isn't enforced — so the attacker forwards (relays) the live exchange to a second, more valuable server in real time, riding the victim's own session.



Practical (isolated AD lab only)

```
# 1. Disable Responder's own SMB/HTTP server so the auth isn't captured-and-dropped
sudo responder -I eth0 --disable-smb --disable-http

# 2. Relay captured auth to a target that doesn't enforce SMB signing
ntlmrelayx.py -tf targets.txt -smb2support

# 3. If successful, you land a shell or dump SAM/secrets on the relay target
```

IMPACT FOR THE REPORT

A successful relay to Domain Admin-equivalent infrastructure (e.g. relaying to a server where the victim has local admin) can go from "captured one hash on the LAN" to full domain compromise in minutes. This is why **Require SMB signing** is one of the highest-value single settings in an AD environment.

Attacking Behind Load Balancers & WAFs

Requested addition

The lecture transcript didn't cover infrastructure like load balancers, but any real-world online password attack (spraying, credential stuffing, brute force against a web login) almost always happens against a **load-balanced, rate-limited, WAF-fronted** endpoint — not a bare server. Here's how that changes your strategy.

9.1 Why load balancers matter to a password attacker

- **Rate limiting is often per-node or per-IP**, not always shared across the whole cluster. Attempts split across backend nodes behind a round-robin load balancer can sometimes reset independent counters — this is exactly why WAFs correlate by client IP/session/cookie, not by backend node, and why testers must check whether the WAF's limiter is centralized before assuming it isn't.
- **Session stickiness (sticky sessions)** can cause inconsistent lockout behavior — a user "locked out" on node A may briefly appear unlocked when routed to node B before state syncs.
- **The X-Forwarded-For / X-Real-IP header** is trusted by many backend apps sitting behind a load balancer to determine the "real" client IP for rate-limiting decisions. If the app trusts this header from any source instead of only from the load balancer itself, an attacker can spoof a different value per request and defeat IP-based throttling entirely.
- **CDN/WAF fingerprinting (Cloudflare, AWS WAF, Akamai, F5 ASM/BIG-IP)** changes your tooling choices — aggressive multi-threaded Hydra runs get fingerprinted and blocked far faster than slow, jittered, browser-like requests.

9.2 Practical strategy for spraying/brute-forcing through a load balancer

1. **Recon the front end first.** Identify the LB/WAF vendor from headers, error pages, and timing (`wafw00f target.com` , `whatwaf`).
2. **Throttle your own tool** to imitate a human, not a scanner — a slow, randomized delay is far less likely to trip a centralized rate limiter than a fast blast.
3. **Test header trust.** In an authorized engagement, send requests with a spoofed `X-Forwarded-For` and observe whether the app's own rate-limit counter resets — this alone is a reportable finding regardless of whether the password attack itself succeeds.
4. **Rotate source context, not identity, carefully.** Only within engagement scope, and only if authorized — this often means using a small pool of client IPs already agreed with the client, never uncontrolled anonymizing infrastructure.
5. **Distinguish "blocked by WAF" from "wrong password."** A WAF-triggered 403/429 is a different signal than an application-level "invalid credentials" — log both separately or your spray results will be full of false negatives.
6. **Prefer protocol-level spraying over the web login where possible** — e.g. spraying against an exposed SMB/LDAP/Kerberos service (Module 5) is often less protected by a WAF than the public-facing web portal, since WAFs are primarily built for HTTP.

Tooling notes

```
# fingerprint the WAF/load balancer in front of a login page
wafw00f https://target.com

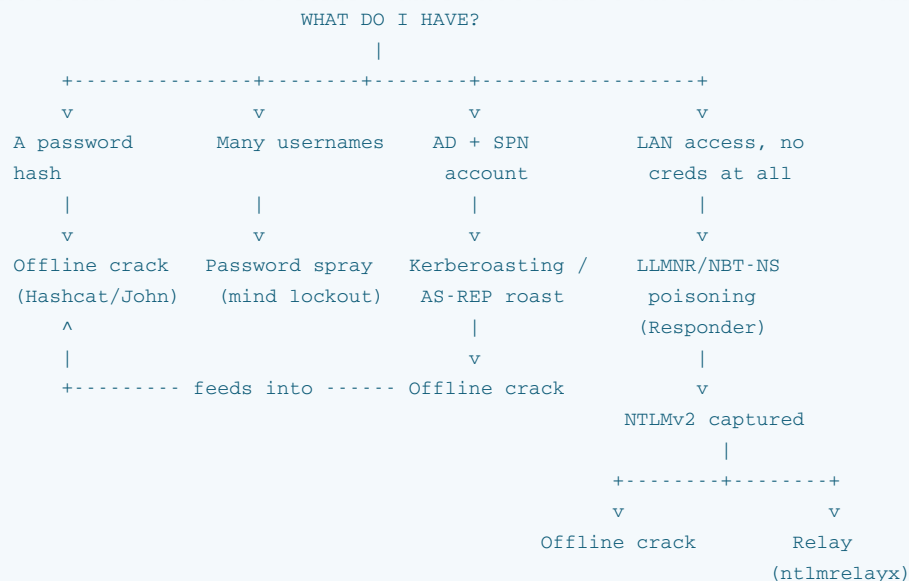
# throttle Hydra to look less like a flood (1 thread, randomized delay via a wrapper script)
hydra -l admin -P rockyou.txt -t 1 -W 5 target.com https-post-form "/login:..."

# test whether the backend trusts a spoofed client-IP header for its rate limiter
curl -s -H "X-Forwarded-For: 1.2.3.4" https://target.com/login -d "user=admin&pass=test1"
```

REPORTING ANGLE

Even when a spray/brute-force attempt fails to find a valid password, "the application's rate limiter can be bypassed via a spoofed X-Forwarded-For header" or "lockout state is inconsistent across load-balanced nodes" are both legitimate, high-value findings on their own.

10.1 "What do I have?" decision tree



10.2 Quick reference table

Technique	Evidence needed	Type	Primary tool	Hashcat mode
Dictionary/mask cracking	Any hash	Offline	Hashcat / John	varies
Password spraying	Username only	Online	NetExec, Kerbrute, Hydra	—
Credential stuffing	Breached user:pass pairs	Online	Hydra (-C)	—
Kerberoasting	Domain creds + SPN account	Online capture → Offline	Impacket / Rubeus	13100
AS-REP roasting	Username, preauth disabled	Online capture → Offline	Impacket GetNPUsers	18200
Responder poisoning	Same LAN segment	Online capture → Offline	Responder	5600
NTLM relay	Same LAN + signing disabled target	Online (live)	ntlmrelayx.py	—
Custom wordlist gen	Target website	Support technique	CeWL	—

Extended Tool Arsenal

Added by mentor

These weren't in the transcript but a junior tester is expected to at least recognize them.

Tool	Category	What it's for
Hydra	Online brute force	Generic protocol brute forcer — HTTP forms, SSH, FTP, RDP, etc.
Medusa	Online brute force	Similar to Hydra, sometimes faster for specific modules
NetExec (formerly CrackMapExec)	AD/SMB toolkit	Spraying, enumeration, executing commands across many hosts at once
Kerbrute	Kerberos	Fast username enumeration + Kerberos-based password spraying (quieter than SMB)
Impacket suite	AD/Windows protocols	GetUserSPNs.py, GetNPUsers.py, secretsdump.py, ntlmrelayx.py, psexec.py
Rubeus	Kerberos (Windows)	Kerberoasting, ticket manipulation, from a compromised Windows host
Mimikatz	Post-exploitation	Extracts plaintext creds/hashes/tickets from memory on a compromised Windows host
BloodHound	AD attack-path mapping	Visualizes who-can-become-Domain-Admin paths; guides which accounts are worth roasting/spraying
Evil-WinRM	Remote access	Use recovered/relayed creds to get an interactive shell over WinRM
CeWL	Wordlist generation	Custom candidate words scraped from a target's own site
hashid / hash-identifier	Recon	Guess a hash's algorithm from its format
wafw00f	Recon	Fingerprint the WAF/CDN in front of a web login

Blue Team / Defensive Playbook

A junior tester who can only attack is half a tester. Every finding needs a fix you can hand to the client. Use this as your standard mitigation checklist when writing up any of the above techniques.

- ☐ Enforce salted, slow hashing (bcrypt/scrypt/argon2) — never unsalted MD5/SHA1 for credential storage.
- ☐ Require strong, unique passwords for service accounts (Kerberoasting mitigation) and rotate them regularly.
- ☐ Enable Managed Service Accounts (gMSA) wherever possible so the password is long, random, and auto-rotated.
- ☐ Disable LLMNR and NBT-NS via Group Policy where legacy name resolution is not required.
- ☐ Enforce SMB signing on all workstations and servers (kills NTLM relay to those hosts).
- ☐ Enforce account lockout thresholds tuned to stop spraying without causing mass self-lockout (e.g. 5-10 attempts / rolling window).
- ☐ Deploy MFA everywhere feasible — it neutralizes most credential-based attacks even if the password is recovered.
- ☐ Monitor for Kerberos ticket requests with unusual encryption types (RC4 requests are a Kerberoasting red flag).
- ☐ Ensure the rate limiter/WAF keys off a properly validated client IP, not a client-supplied header like X-Forwarded-For.
- ☐ Centralize authentication rate-limit state across all load-balanced nodes, not per-node counters.

Reporting Template

Copy this structure for every credential-access finding in your report — it's what separates a "script kiddie log" from a professional deliverable.

```
Finding:          <short title, e.g. "Weak Service Account Password via Kerberoasting">
Severity:         <Critical / High / Medium / Low, per your engagement's rating scale>
Evidence:         <exactly what you obtained, and how, with timestamps>
Technique:        <e.g. Offline dictionary attack against a Kerberos TGS ticket>
Tool(s):          <e.g. Impacket GetUserSPNs.py, Hashcat -m 13100>
Wordlist/rules:   <e.g. rockyou.txt + best64.rule>
Result:           <password recovered / relay succeeded / spray hit, etc.>
Impact:           <what an attacker could do next with this credential/access>
Recommendation:  <concrete, prioritized fix — see Module 12 checklist>
```

Quiz Answer Key

14.1 Lesson 1 quiz

Q1. Online vs. offline attacks — the difference?

Online: every guess is sent to the live server and the server decides accept/reject. Offline: you already possess a hash/ticket/capture and test candidates locally — the server sees nothing.

Q2. What does Hashcat do with a wordlist against a hash?

It hashes each candidate word from the list using the specified algorithm and compares the result to the target hash, reporting a match as the recovered password.

Q3. Dictionary vs. brute force — which is more efficient against a human-chosen password?

Dictionary attack — humans pick predictable, reused patterns, so a curated wordlist finds the password far faster than exhaustively generating every possible combination.

Q4. What does `-a 0` mean in a Hashcat command?

Attack mode 0 = straight dictionary attack (as opposed to `-a 3` brute-force/mask, or `-a 6/7` hybrid).

14.2 Lesson 6 quiz (NTLM/Responder)

Q1. What does NTLM use instead of sending the password directly?

A challenge-response exchange — the client proves knowledge of the password-derived secret without transmitting the password itself.

Q2. What are LLMNR and NBT-NS used for?

Fallback local-network name resolution when normal DNS fails to resolve a hostname.

Q3. Responder's role?

It answers LLMNR/NBT-NS requests, impersonating the requested host, tricking the victim into authenticating to the attacker and capturing the NTLMv2 challenge-response.

Q4. After capturing NTLMv2 material, what can the attacker do?

Attempt offline cracking (Hashcat `-m 5600`) or relay the live exchange to another service (NTLM relay).

Q5. Crack vs. relay?

Crack = recover the plaintext password offline, no time pressure. Relay = forward the live authentication session in real time to a second target, no password recovery needed.

Q6. Complete the chain: Victim → LLMNR/NBT-NS → ____ → NTLMv2 challenge-response → ____

Victim → LLMNR/NBT-NS → **Responder** → NTLMv2 challenge-response → **offline cracking or relay**.

14.3 Lesson 7/8 decision exercises

Case	Correct technique
Hash + rockyou.txt	Dictionary attack (offline)
500 usernames, limited attempts allowed	Password spraying
AD service account with an SPN	Kerberoasting
LLMNR traffic + NTLM auth observed on LAN	Responder

Compiled for study purposes — always operate within written authorization and applicable law.